TASKFLOW API · SERVER/

Backend Guide

Node.js · Express · Neon PostgreSQL
Architecture, API, sync, notifications and deployment

TaskFlow API 1.0 · Node 22 · TypeScript 5.9 · Express 5 · Drizzle ORM · Neon

Contents

1. Overview 4
What it does 4
Technology stack 5
Project layout 5

2. Getting started 6
Create the Neon database 6
Configure and run 6
Environment variables 7
npm scripts 7

3. Architecture 8
Request lifecycle 8
Composition root 8
Transactions and per-user ordering 8
Why the Neon serverless driver 9

4. Modules 10
Adding a new module 10

5. Database 11
Tables 11
The change cursor 11
Indexes 12
Migrations 12

6. API reference 13
Conventions 13
Endpoints 14
Example: sign up and create a task 15

7. Authentication and security 16
Tokens 16
Refresh rotation and reuse detection 16
Passwords 16
HTTP hardening 16

8. Sync protocol 17

Push 17

Pull 18

Conflict rules 18

9. Notifications and background jobs 19

Who shows what 19

Jobs 19

10. Deployment 20

Docker 20

Firebase setup 20

Hosting checklist 20

Continuous integration 20

11. Testing and quality 21

Troubleshooting 21

1. Overview

The `server/` folder holds the TaskFlow API: accounts, tasks, offline-first sync, settings, device registration, push notifications and stats for the Flutter app. It is written in TypeScript with the same feature-first clean architecture the app uses, and stores data in Neon (serverless PostgreSQL).

What it does

- **Accounts**: register, log in, refresh and log out with short-lived JWT access tokens and rotating refresh tokens. Profile, password change and full account deletion.
- **Tasks**: a REST API for tasks and subtasks (create, update, complete with repeat rules, search and filters), used by tools and future clients.
- **Sync**: the app works offline and exchanges changes through two endpoints, `POST /sync/push` and `GET /sync/pull`, with a global change cursor and last-write-wins.
- **Notifications**: the app shows reminders locally. The server adds Firebase push for what a phone cannot know yet (a task changed on another device), a morning digest, instant "sync" nudges, and an inbox.
- **Stats**: flow score, on-time rate, streaks and category breakdown computed in SQL, in the user's time zone.
- **Docs**: a full OpenAPI 3.1 document generated from the validation schemas, with Swagger UI at `/docs`.

Technology stack

Area	Choice	Why
Runtime	Node.js 22 LTS, TypeScript 5.9 (strict, ESM)	Type safety end to end; modern ESM modules.
HTTP	Express 5	Native async error handling, mature ecosystem.
Database	Neon PostgreSQL	Serverless Postgres with branching, autoscaling and a pooled endpoint.
DB driver	`@neondatabase/serverless` (WebSocket Pool); `pg` for other hosts	Full transactions over WebSockets on Neon; `DB_DRIVER=auto` picks the right one.
ORM / migrations	Drizzle ORM + drizzle-kit	SQL-first, typed queries, generated SQL migrations.
Validation & docs	zod 4 + zod-to-openapi	One schema validates requests and generates OpenAPI.
Auth	jose (JWT HS256), Argon2id (`@node-rs/argon2`)	Standards-based tokens; memory-hard password hashing.
Security	helmet, cors, express-rate-limit	Secure headers, origin control, brute-force protection.
Push	firebase-admin (FCM HTTP v1)	Android and iOS (through APNs) with one API.
Time	luxon	Correct time zones and DST for due dates, reminders and digests.
Logging	pino + pino-http	Fast structured JSON logs with request ids.
Tests	Vitest + supertest + PGlite	Real Postgres (in-process WASM) for integration tests, no Docker needed.

Project layout

Folder structure

```
server/
  src/
    server.ts              HTTP entry point (graceful shutdown)
    worker.ts              optional separate process for background jobs
    app.ts                 Express app: middleware, routers, OpenAPI, error handler
    container.ts           composition root: wires repositories, services, jobs
    config/env.ts          environment variables, validated with zod
    core/                  shared building blocks (no business rules)
      db/                  schema, client (Neon / pg), unit of work, migrate
      http/                zod + OpenAPI helpers, validation, middleware
      security/            password hasher, token service
      jobs/scheduler.ts    interval scheduler for background jobs
      errors/, logger.ts, utils/clock.ts
    modules/               one folder per feature
      auth/ users/ tasks/ sync/ settings/ devices/ notifications/ stats/ health/
        domain/            entities, rules, repository interfaces
        application/       services (use cases) and request/response schemas
        infrastructure/    Drizzle repositories, gateways (FCM)
        presentation/      Express routes and OpenAPI registration
    jobs/cleanup.job.ts    housekeeping
  drizzle/                 generated SQL migrations
  tests/                   unit + integration tests (PGlite)
  docs/openapi.json        exported API description
  Dockerfile, docker-compose.yml, .env.example
```

2. Getting started

From a fresh checkout to a running API against Neon in a few minutes. You need Node.js 22+ and a free Neon account.

Create the Neon database

1. Sign in at **console.neon.tech** and create a project (pick the region closest to where the API will run).
2. Open **Connect**. Copy the **pooled** connection string (the host contains `-pooler`) and the **direct** one (toggle "Connection pooling" off).
3. Keep `?sslmode=require` at the end of both strings.

> **Why two strings.** The API uses the pooled string (`DATABASE_URL`) because it opens many short connections. Migrations use the direct string (`MIGRATION_DATABASE_URL`): they need a session-level lock, which the pooler (PgBouncer in transaction mode) does not keep.

Configure and run

Terminal

```
cd server
cp .env.example .env        # fill in DATABASE_URL, MIGRATION_DATABASE_URL, JWT_ACCESS_SECRET
npm install
npm run db:migrate          # creates the tables in Neon
npm run dev                 # http://localhost:8080  ·  API docs at http://localhost:8080/docs
```

Generate the JWT secret with `node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"`. The server refuses to start if a variable is missing or invalid and prints which one.

Environment variables

Variable	Default	Meaning
DATABASE_URL	required	Pooled Neon connection string used by the API.
MIGRATION_DATABASE_URL	= DATABASE_URL	Direct (non-pooled) string for migrations.
DB_DRIVER	auto	neon (serverless driver), pg (node-postgres) or auto (Neon for *.neon.tech).
DB_POOL_MAX	10	Maximum connections per process.
JWT_ACCESS_SECRET	required	At least 32 characters. Signs access tokens.
JWT_ACCESS_TTL_SECONDS	900	Access token lifetime (15 minutes).
REFRESH_TOKEN_TTL_DAYS	30	Refresh token lifetime; each use rotates it.
PORT / NODE_ENV / LOG_LEVEL	8080 / development / info	Standard process settings.
CORS_ORIGINS	*	Comma-separated list of allowed browser origins.
TRUST_PROXY	1	Hops of reverse proxies to trust for client IPs (rate limits).
DOCS_ENABLED	true	Serve Swagger UI at /docs and JSON at /openapi.json.
JOBS_ENABLED	true	Run reminders, digest and cleanup in the API process.
REMINDER_INTERVAL_MS	30000	How often due reminders are checked.
DIGEST_INTERVAL_MS	60000	How often digests are checked.
CLEANUP_INTERVAL_MS	3600000	Housekeeping interval (hourly).
FIREBASE_SERVICE_ACCOUNT	empty	Service account JSON (raw or base64). Empty = pushes are logged only.

npm scripts

Script	What it does
npm run dev	Starts the API with hot reload (tsx watch), loading .env.
npm run dev:worker	Starts the background-job worker with hot reload.
npm run build / start	Compiles to dist/ and runs the compiled server.
npm run db:generate	Creates a new SQL migration from changes in schema.ts.
npm run db:migrate	Applies pending migrations (uses MIGRATION_DATABASE_URL).
npm run db:studio	Opens Drizzle Studio to browse the database.
npm test / test:coverage	Runs all tests (and coverage) against in-process Postgres.
npm run check	Type check + lint + tests: what CI runs.
npm run openapi:export	Writes docs/openapi.json.

3. Architecture

Each feature is a vertical slice with four layers. Dependencies point inwards: routes call services, services use domain rules and repository interfaces, and the infrastructure implements those interfaces with Drizzle and Firebase. This mirrors the app's data, domain and presentation split.

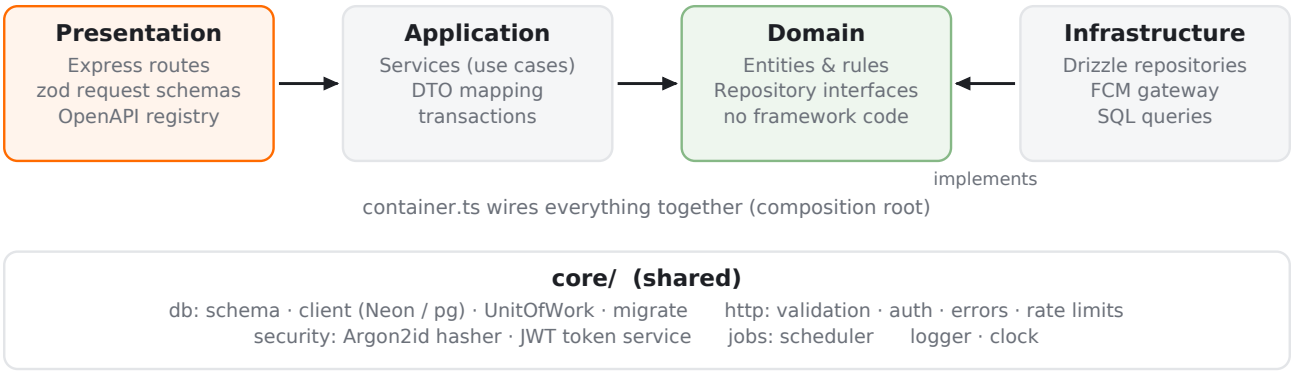


Figure 1. Layers of a module. Arrows show the direction of dependencies.

Request lifecycle

1. **Request context**: assigns a request id (or reuses `X-Request-Id`), reads the optional `X-Device-Id` header, and creates a child logger.
2. **Security middleware**: helmet headers, CORS, compression, JSON body limit, rate limits.
3. **Authenticate**: verifies the Bearer JWT (signature, issuer, audience, expiry) and puts `req.auth.userId` on the request.
4. **Validate**: the route parses `body`, `query` and `params` with zod. Invalid input becomes a 422 with a list of field errors.
5. **Service**: runs the use case. Writes happen inside a unit of work (one transaction).
6. **Response**: success returns `{ "data": ... }`. Any thrown error goes to the error handler and becomes an RFC 9457 `application/problem+json` body.

Composition root

`container.ts` is the only place that knows concrete classes. It builds the database, repositories, the push gateway (FCM when `FIREBASE_SERVICE_ACCOUNT` is set, otherwise a logging gateway), services and jobs, and hands them to `createApp()`. Tests build the same container with an in-process database, a fixed clock and an in-memory push gateway, so integration tests exercise the real code paths.

Transactions and per-user ordering

Services receive a `UnitOfWork`. `uow.run(tx => ...)` opens a transaction and passes an opaque `TxContext` to repositories, so the application layer never imports Drizzle. Every write for a user first takes `pg_advisory_xact_lock(hashtextextended(userId, 0))`. Writes from two devices of the same user are therefore serialised, which keeps the change cursor and last-write-wins decisions consistent, while different users never wait on each other.

Why the Neon serverless driver

The `@neondatabase/serverless` Pool talks to Neon over WebSockets, supports interactive transactions (needed for the unit of work) and works from serverless and edge runtimes as well as from a normal server. For local Postgres (docker compose) `DB_DRIVER=auto` switches to node-postgres, so the code is identical everywhere.

4. Modules

Each module owns its routes, rules and tables. Modules talk to each other only through interfaces passed in by the container.

Module	Responsibility	Key pieces
auth	Register, login, refresh rotation, logout.	`AuthService`, `RefreshToken` domain, user and refresh-token repositories.
users	The signed-in account: profile, name change, password change, delete account.	`AccountService`; deleting cascades to every table.
tasks	Task aggregate (task + subtasks), schedule maths, repeat rules, REST routes.	`task.ts`, `schedule.ts` (due and reminder instants in the task's zone), `repeat-rule.ts`, `task-writer.ts`, `TaskService`.
sync	Offline-first exchange: push local changes, pull remote ones.	`SyncService`, `ChangeNotifier` interface (implemented by notifications).
settings	Preferences (theme, digest, goal, categories, time zone) with last-write-wins.	`SettingsService`; defaults stamped with the epoch so a first upload always wins.
devices	One row per app install: platform, push token, time zone, last pulled cursor.	`DeviceService`, `PUT/DELETE /devices/{installationId}`.
notifications	Push reminders, digest, sync nudges, inbox, test push.	`ReminderDispatcher`, `DigestDispatcher`, `SyncPushNotifier`, `PushDispatcher`, FCM and log gateways.
stats	Productivity summary for a period.	`StatsService`, `productivity.ts` (pure scoring).
health	Liveness and readiness probes.	`/health/live`, `/health/ready` (checks the DB).

Adding a new module

1. Create `src/modules/<name>/{domain,application,infrastructure,presentation}`.
2. Domain: entity types and a repository interface (no imports from Drizzle or Express).
3. Tables: add them to `core/db/schema.ts`, then run `npm run db:generate` and review the SQL.
4. Infrastructure: a `Drizzle<Name>Repository` implementing the interface.
5. Application: zod schemas (`.openapi('Name')`) and a service that uses the repository.
6. Presentation: a router that validates input, calls the service and registers its paths.
7. Wire the repository and service in `container.ts` and mount the router in `app.ts`.
8. Add integration tests in `tests/integration/` using `createTestApp()`.

5. Database

Seven tables in PostgreSQL. Every id is a UUID. Tasks use ids created on the phone, so they can be made offline and uploaded later without conflicts.

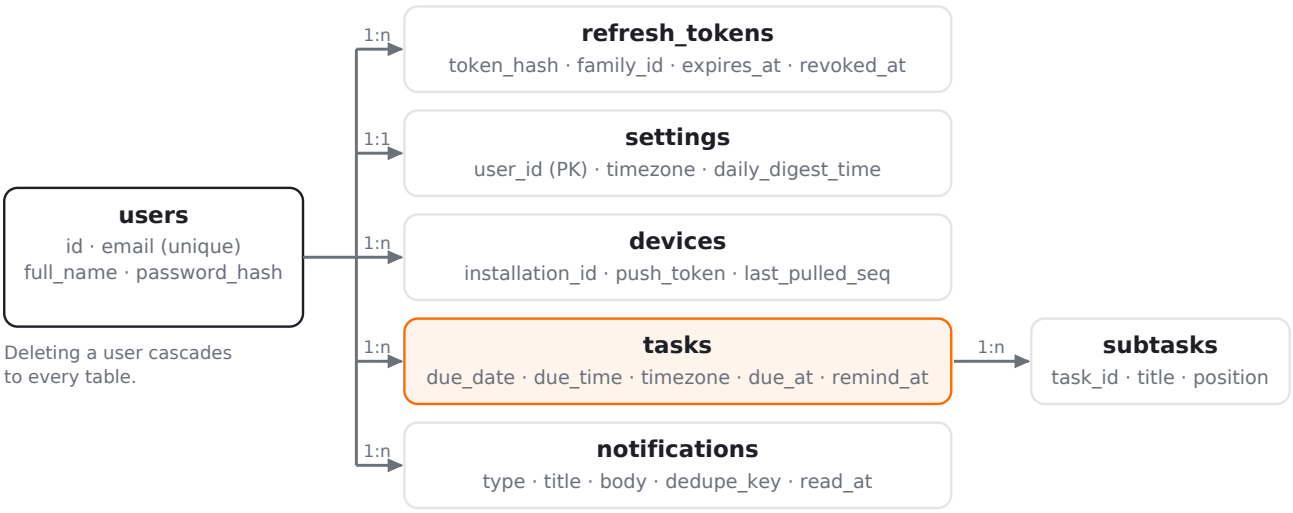


Figure 2. Tables and relations.

Tables

Table	Purpose and notable columns
users	Accounts. `email` is stored lower-cased with a unique index (case-insensitive). `password_changed_at` revokes older sessions.
refresh_tokens	SHA-256 hash of each opaque refresh token (the token itself is never stored), `family_id` (one per sign-in), `replaced_by_id`, `revoked_at`, device and IP for auditing.
settings	One row per user. Check constraints on theme, priority and daily goal. `updated_at` is the client time of the last edit (last-write-wins).
tasks	Calendar day, time and IANA zone as entered, plus derived `due_at` and `remind_at` instants (all-day tasks are due at the end of the day and remind at 9 AM, both in the task's zone). `deleted_at` is the sync tombstone. `reminder_sent_at` makes the push fallback fire once per reminder time.
subtasks	Ordered by `position`; replaced as a whole with their task.
devices	Unique per (user, installation). Push tokens are unique across users (a phone that switches accounts moves its token). `last_pulled_seq` tells the server what the phone already knows.
notifications	Inbox of reminders, digests and test pushes. `dedupe_key` (unique) makes every job idempotent.

The change cursor

A global sequence `change_seq` stamps every write to `tasks` and `settings` (a trigger-free default of `nextval('change_seq')`, refreshed on each update). A pull returns rows whose `change_seq` is greater than the client's cursor, ordered by it, and the largest value becomes the new cursor. The index `(user_id, change_seq)` makes this a range scan.

Indexes

- `tasks_user_change_seq_idx (user_id, change_seq)`: sync pulls.
- `tasks_user_due_idx (user_id, due_at)`: task lists, calendar and stats.
- `tasks_pending_reminder_idx (remind_at) WHERE reminder AND NOT completed AND deleted_at IS NULL AND reminder_sent_at IS NULL`: a small partial index the reminder job scans every 30 s.
- `notifications_dedupe_key` (unique) and `devices_push_token_key` (unique, partial).

Migrations

The schema is defined in TypeScript (`core/db/schema.ts`). After changing it run `npm run db:generate`; drizzle-kit writes a SQL file to `drizzle/` that you review and commit. `npm run db:migrate` (or `db:migrate:prod` in the container) applies pending files in order and records them, so it is safe to run on every deploy.

> **Tip.** Neon branches are ideal for testing a migration: create a branch from production in the console, point `MIGRATION_DATABASE_URL` at it, migrate, test, then run it on main.

6. API reference

Base path `/api/v1`. Full request and response schemas, with examples, are in Swagger UI at `/docs` and in `docs/openapi.json`.

Conventions

- **Auth**: `Authorization: Bearer <accessToken>` on everything except register, login, refresh and health.
- **Device**: the app sends `X-Device-Id: <installation id>` so the server can skip the sending device when it notifies the others and can track sync progress.
- **Success**: `200/201` with `{ "data": ... }`; `204` with no body for deletes.
- **Errors**: RFC 9457 problem details with a stable `code`, a human `detail`, the `requestId`, and `errors[]` (field + message) for validation failures.
- **Dates**: instants are ISO 8601 UTC. Due dates are `YYYY-MM-DD` and times `HH:mm` in the task's `timezone`.
- **Rate limits**: 600 requests/min per IP overall, 20/min per IP on auth routes, 120/min per user on sync, 5/min for the test push. Standard `RateLimit` headers are returned.

Error response (example)

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://taskflow.app/problems/validation_failed",
  "title": "Unprocessable Content",
  "status": 422,
  "detail": "The request body is invalid.",
  "code": "validation_failed",
  "requestId": "4f1c2a...",
  "errors": [
    { "path": "password",
      "message": "Use 8+ characters with upper and lower case and a number." }
  ]
}
```

Endpoints

Method	Path	Description
GET	/health/live	Liveness probe (no database).
GET	/health/ready	Readiness probe (checks the database).
POST	/auth/register	Create an account; returns user + tokens.
POST	/auth/login	Sign in with email and password.
POST	/auth/refresh	Exchange a refresh token for a new pair (rotation).
POST	/auth/logout	Revoke the session of a refresh token.
GET	/users/me	The signed-in user.
PATCH	/users/me	Update the name.
DELETE	/users/me	Delete the account and all its data (password required).
POST	/users/me/password	Change the password; other sessions are signed out.
GET	/tasks	List with `status`, `from`, `to`, `q`, `priority`, `category`, `limit`, `cursor`.
POST	/tasks	Create a task (optionally with a client id).
GET	/tasks/{id}	One task with subtasks.
PATCH	/tasks/{id}	Partial update.
DELETE	/tasks/{id}	Delete (becomes a tombstone for sync).
POST	/tasks/{id}/complete	Complete; a repeating task gets its next occurrence.
POST	/tasks/{id}/reopen	Reopen a completed task.
POST	/tasks/{id}/subtasks	Add a subtask.
PATCH	/tasks/{id}/subtasks/{subtaskId}	Rename or tick a subtask.
DELETE	/tasks/{id}/subtasks/{subtaskId}	Remove a subtask.
POST	/sync/push	Upload local changes (tasks and settings).
GET	/sync/pull	Changes since a cursor, paged.
GET	/settings	Preferences.
PATCH	/settings	Change preferences.
PUT	/devices/{installationId}	Register or update this install and its push token.
DELETE	/devices/{installationId}	Unregister (on sign-out).
GET	/notifications	Inbox of reminders and digests.
POST	/notifications/{id}/read	Mark one as read.
POST	/notifications/read-all	Mark all as read.
POST	/notifications/test	Send a test push to all my devices.
GET	/stats/summary	Flow score, on-time rate, streaks; `period` = week, month, all.

Health routes live at the root (`/health/...`); every other path is under `/api/v1`.

Example: sign up and create a task

Terminal

```
curl -X POST http://localhost:8080/api/v1/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"fullName": "Alex Morgan", "email": "alex@example.com",
       "password": "Secret123", "timezone": "Asia/Colombo"}'

# -> { "data": { "user": { ... },
#                "tokens": { "accessToken": "eyJ...", "refreshToken": "...",
#                            "tokenType": "Bearer", "expiresIn": 900 } } }

curl -X POST http://localhost:8080/api/v1/tasks \
  -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
  -d '{"title": "Send the report", "dueDate": "2026-10-02", "dueTime": "14:00",
       "timezone": "Asia/Colombo", "priority": "high",
       "reminder": true, "reminderMinutes": 30}'
```

7. Authentication and security

Short-lived access tokens, rotating refresh tokens with theft detection, memory-hard password hashing and defensive HTTP defaults.

Tokens

Token	Format	Lifetime	Storage
Access	JWT, HS256, `sub` = user id, issuer and audience checked	15 minutes	Phone: secure storage (or memory only without "Remember me"). Server: not stored.
Refresh	32 random bytes (base64url), opaque	30 days, rotated on every use	Phone: Keychain / Keystore. Server: SHA-256 hash only.

Refresh rotation and reuse detection

1. Every login starts a new token **family**.
2. `POST /auth/refresh` revokes the presented token, issues a new one in the same family and links them (`replaced_by_id`).
3. If an already-rotated token is presented again **after** a 30-second grace window, the server assumes it was stolen and revokes the whole family: every device using that session must sign in again.
4. Within the grace window (two requests racing on a flaky network) the request is rejected without revoking, so honest clients are not signed out.
5. Changing the password or deleting the account revokes every refresh token of the user.

Passwords

Passwords are hashed with **Argon2id** (19 MiB memory, 2 iterations, parallelism 1: the OWASP baseline) using the native `@node-rs/argon2` binding. Rules: 8+ characters with upper case, lower case and a number, up to 128. Login returns the same error for an unknown email and a wrong password, and still runs a hash for unknown emails so timing does not reveal which accounts exist.

HTTP hardening

- `helmet` sets secure headers; `x-powered-by` is off.
- CORS is limited to `CORS_ORIGINS` (mobile apps are not affected by CORS).
- JSON bodies are limited to 1 MB; every input is validated and unknown enum values rejected.
- Rate limits per IP (auth) and per user (sync) slow down brute force and runaway clients.
- All queries are parameterised by Drizzle; every query is scoped by `user_id`, so one user can never read or change another user's rows (tests cover this).
- 500 errors never leak internals: details are logged with the request id, the client gets a generic message and the id to report.
- Secrets come only from the environment; `.env` is git-ignored.

8. Sync protocol

The app keeps its own SQLite database and exchanges changes with the server. Two endpoints, a cursor, tombstones and last-write-wins keep every device consistent even after days offline.

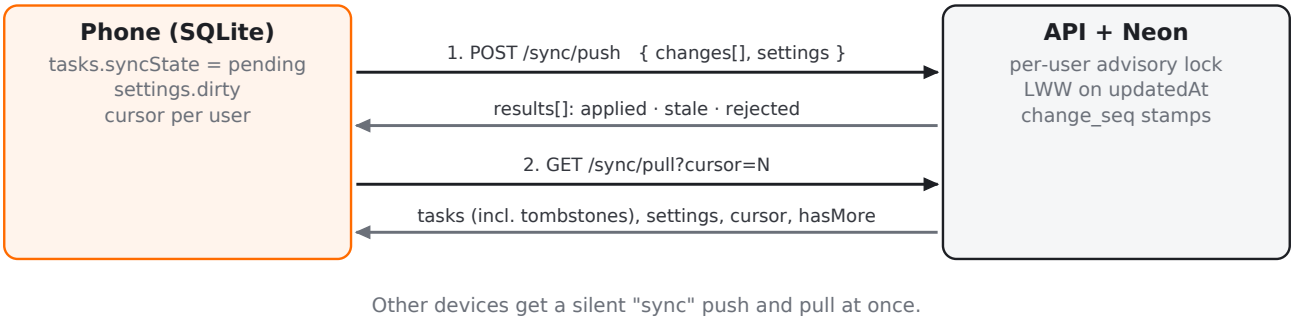


Figure 3. One sync round: push first, then pull until hasMore is false.

Push

The client sends up to 200 changes per request. A change is either `{ "op": "upsert", "task": {...whole task with subtasks...} }` or `{ "op": "delete", "id": "...", "deletedAt": "..." }`, plus optional `settings`. Each change is validated and applied on its own, so one bad row can never block the queue:

Result	Meaning	Client action
`applied`	Stored; it gets a new change_seq.	Mark the row synced.
`stale`	The server already has a newer version (by `updatedAt`) or a newer tombstone.	Mark synced; the newer version arrives with the pull.
`rejected`	Invalid (e.g. text too long). A `reason` explains why.	Mark synced; do not retry.

Push request and response

```
POST /api/v1/sync/push
{
  "changes": [
    { "op": "upsert", "task": {
        "id": "9b2e...", "title": "Send the report", "description": "",
        "dueDate": "2026-10-02", "dueTime": "14:00", "isAllDay": false,
        "timezone": "Asia/Colombo", "priority": "high", "category": "Work",
        "isCompleted": false, "completedAt": null,
        "reminder": true, "reminderMinutes": 30, "repeat": "none",
        "subtasks": [{ "title": "Charts", "isCompleted": true }],
        "createdAt": "2026-09-29T08:00:00.000Z", "updatedAt": "2026-09-29T08:05:12.000Z" } },
    { "op": "delete", "id": "c41d...", "deletedAt": "2026-09-29T08:06:00.000Z" }
  ],
  "settings": { "themeMode": "dark", "dailyGoal": 5, "updatedAt": "2026-09-29T08:01:00.000Z" }
}

-> { "data": {
       "results": [ { "id": "9b2e...", "status": "applied" },
                    { "id": "c41d...", "status": "applied" } ],
       "settings": "applied" } }
```

Pull

`GET /sync/pull?cursor=N&limit=500` returns tasks whose `change_seq > N` (deleted ones included, with `deletedAt` set), the settings if they changed, the new `cursor` and `hasMore`. The client repeats until `hasMore` is false and saves the cursor. With `X-Device-Id` the server records the cursor as the device's `last_pulled_seq`, which the reminder job uses to decide whether a phone already knows about a change.

Conflict rules

- **Tasks**: last write wins on the client's `updatedAt`: an incoming change is applied unless the stored version is strictly newer. A task and its subtasks travel together, so they can never be half-merged.
- **Deletes** are tombstones and beat older edits; an edit made after the deletion wins over it. A tombstone is purged 30 days after deletion.
- **Settings**: last write wins on `settings.updatedAt`. Defaults on a new account are dated 1970, so the settings a phone had before signing up always win on first sync.
- **On the phone**: a pulled task is skipped if the local copy has unsent changes that are newer.

9. Notifications and background jobs

Reminders are local first: the app schedules them on the phone, where they fire even offline. The server covers the gaps with Firebase Cloud Messaging and never sends the same reminder twice.

Who shows what

Notification	Phone (local)	Server (push)
Task reminder	Scheduled for every task with a reminder on this device.	Sent only to devices that do not know the current version of the task yet (`last_pulled_seq` < task's `change_seq`) or that do not schedule reminders locally.
Daily digest	Scheduled locally at the chosen time.	Only for devices registered with `localReminders = false`.
Sync nudge	-	Silent `{ type: "sync" }` data push to the user's other devices after a change, coalesced over 2 seconds.
Test	-	`POST /notifications/test` sends one to every device of the user.

Jobs

Job	Every	What it does
`reminders`	`30 s`	Claims due reminders with `FOR UPDATE SKIP LOCKED` (safe with several instances), marks them sent, writes an inbox entry with a dedupe key, then pushes. Reminders more than 1 hour late are skipped instead of arriving out of context.
`digest`	`60 s`	For users whose digest time has come in their own time zone, sends a summary of today's open tasks once per day (dedupe key per user and date).
`cleanup`	`1 h`	Deletes refresh tokens expired or revoked over 7 days ago, tombstones older than 30 days, inbox entries older than 90 days and devices unseen for 180 days.

Jobs run inside the API process by default (`JOBS_ENABLED=true`). For larger deployments, set `JOBS_ENABLED=false` on the API instances and run one worker process (`npm run start:worker`). Firebase setup is in chapter 10. Delivery is at-most-once: a reminder is marked sent before pushing, so a crash can drop one push but never duplicates it (the local reminder still fires on the phone).

10. Deployment

The API is a stateless container. State lives in Neon; the only in-memory state is the rate-limit counters.

Docker

```
Terminal
docker build -t taskflow-api ./server
docker run -p 8080:8080 --env-file server/.env taskflow-api
# The image runs migrations, then starts the server. Health check: GET /health/live
```

`docker compose up` in `server/` starts a local PostgreSQL 17 and the API together, which is handy for working without Neon.

Firebase setup

1. Create a project at **console.firebase.google.com** and add an Android app (package `com.taskflow.taskflow`) and an iOS app (bundle id `com.taskflow.taskflow`).
2. Server: **Project settings > Service accounts > Generate new private key**. Put the JSON in `FIREBASE_SERVICE_ACCOUNT` (one line, or `base64 -w0 key.json`).
3. iOS: upload an **APNs authentication key** in **Project settings > Cloud Messaging**.
4. App: pass the client values with `--dart-define-from-file` (see the Frontend guide).
5. Check it: sign in on a phone, then `POST /api/v1/notifications/test`.

Without a service account the server uses a logging gateway: everything works and each push is written to the log instead. Tokens that FCM reports as invalid are removed automatically.

Hosting checklist

- Any container host works (Render, Railway, Fly.io, Google Cloud Run, AWS App Runner, a VPS). Choose a region close to the Neon region.
- Set `NODE_ENV=production`, a strong `JWT_ACCESS_SECRET`, the two Neon strings, `CORS_ORIGINS` and `FIREBASE_SERVICE_ACCOUNT`.
- Serve over HTTPS (the host's proxy) and set `TRUST_PROXY` to the number of proxies.
- Point the readiness check at `/health/ready` and liveness at `/health/live`.
- Consider `DOCS_ENABLED=false` in production if the API should not be browsable.
- Scaling out: run one worker for jobs (or keep jobs on; `SKIP LOCKED` prevents double work) and plug a shared store into the rate limiter (see `rate-limit.ts`).
- Graceful shutdown: on SIGTERM the server stops accepting connections, lets in-flight requests finish, stops the jobs, flushes pending sync pushes and closes the pool (forced exit after 10 s).

Continuous integration

`.github/workflows/ci.yml` runs `npm ci`, type check, lint, tests and build for the server, and `flutter pub get`, `flutter analyze` and `flutter test` for the app, on every push and pull request.

11. Testing and quality

Tests run against a real PostgreSQL engine (PGlite, in-process) with the real migrations, so SQL, constraints and transactions are exercised exactly as in production. No Docker or Neon account is needed.

Suite	Covers
`unit/domain.test.ts`	Schedule maths across time zones and daylight saving, all-day tasks, repeat rules, the task writer, productivity scoring and streaks.
`integration/auth.test.ts`	Sign-up with default settings, duplicate email (409) and weak passwords, login that never reveals which part was wrong, refresh rotation and replay revocation, logout, expired tokens, profile / password / account deletion, problem details, health and OpenAPI.
`integration/tasks.test.ts`	Derived due and reminder instants, validation, filters, search and keyset paging, user isolation, rescheduling, repeating tasks completed exactly once, subtasks, client ids, stats.
`integration/sync.test.ts`	Offline work pushed and pulled on another device, paging, last-write-wins, tombstones, bad changes rejected one by one, id conflicts, settings, silent sync pushes.
`integration/notifications.test.ts`	Reminder targeting by device state, re-sending after a reminder moves, the notifications switch, dead tokens, the daily digest, cleanup.

Current result: **47 tests passing**, with type check, ESLint and Prettier clean. `tests/helpers/test-app.ts` builds the full app with a fixed clock and an in-memory push gateway, so time-based behaviour is deterministic.

Troubleshooting

Symptom	Fix
Server exits with "Invalid environment"	The message lists the variables; compare with `.env.example`.
Migration hangs or fails on Neon	Use the direct (non-pooler) string in `MIGRATION_DATABASE_URL`.
App on an Android emulator cannot connect	Use `http://10.0.2.2:8080/api/v1`; on a real phone use the computer's LAN IP or an HTTPS URL.
401 after 15 minutes	Expected: the app refreshes automatically. For curl, call `/auth/refresh`.
Pushes are only logged	`FIREBASE_SERVICE_ACCOUNT` is empty or invalid; check the startup log.
No push on iOS	APNs key missing in Firebase or the Push capability not enabled in Xcode.
429 Too Many Requests	A rate limit was hit; wait for the window in the `RateLimit` header.